Executing Author JavaScript in Educational Resources: A Security Evaluation of Isolation in Moodle, WordPress, Omeka S, SCORM, H5P, and eXeLearning

Ernesto Serrano · info@ernesto.es

2026-06-14

Índice

Independent Researcher, Spain · ORCID: 0009-0006-3817-1317 · info@ernesto.es

Personal capacity. Conflict-of-interest disclosure: the author is a collaborator of the eXeLearning project and author/maintainer of several evaluated pieces (`mod_exelearning`, `wp-exelearning`, `omeka-s-exelearning`, and `wp-framer`); the mitigation in Section 6.2 is the author's own contribution. See Section 9.

> **Skeleton (English version).** Structure, abstract, research questions, and the two core tables are complete; prose sections carry one-line stubs to be expanded from the Spanish paper (`seguridad-html-js-recursos-educativos.md`). Shared bibliography: `references.bib`.

Abstract

Interactive educational resources —SCORM, H5P, eXeLearning packages, HTML pages— often need JavaScript to work. The problem is not JavaScript: it is executing **untrusted** JavaScript inside an authenticated LMS session without an explicit isolation boundary. This is an **empirical security evaluation** of nine common ways of publishing content in Moodle, WordPress, and Omeka S, conducted with the source code in hand and an innocuous capability probe run in a local lab. The central finding, organised in a **comparative matrix** under a single mental model (does it run author JS? same origin as the platform? is there real isolation?), is that when content runs in the **same origin** as the platform it can read the authenticated DOM, token-bearing forms, and ride the session; when **isolated** (opaque origin, `sandbox` without `allow-same-origin`, or `srcdoc`), it cannot.

We distinguish three states precisely: **(a)** the *analysed stable releases* of the eXeLearning integrations and Moodle's native SCORM run content in the **same origin**; **(b)** the *maintained eXeLearning integrations* ship an **opaque-origin secure mode enabled by default** that closes this exposure; **(c)** `mod_page`, native SCORM, and `mod_exeweb`/`mod_exescorm` remain *same-origin* by design. H5P is a separate case: it does not execute author

HTML, only curated libraries. On `mod_page` we correct a common misconception: **its real protection is not server-side sanitisation (it uses `noclean=true`) but capability/role restriction**. Generative AI worsens the risk by making it trivial to paste unreviewed HTML/JS; a public Spanish education body (CEDEC/INTEF) has warned that unreviewed AI-generated code in Open Educational Resources undermines both openness and safety [1]. We present and browser-verify a hardening pattern —opaque origin + validated `postMessage` bridge + CSP— that preserves interactivity and tracking without treating content as part of the platform.

> **Thesis:** the primary risk of interactive educational resources is not JavaScript, but executing **author** JavaScript within the authenticated same origin of the LMS. A model based on opaque origin, strict `sandbox`, CSP, and a validated `postMessage` bridge keeps interactivity without trusting content as if it were part of the platform.

Keywords: web security; LMS; Moodle; eXeLearning; SCORM; H5P; WordPress; Omeka S; iframe isolation; same-origin policy; `sandbox`; Content Security Policy; `postMessage`; XSS; AI-generated content.

1. Introduction

> *Stub (expand from ES Section 1):* problem framing (interactive content + generative AI + authenticated LMS), motivation, the OER/REA angle (CEDEC/INTEF).

Research question. To what extent do the common ways of publishing interactive educational resources in Moodle, WordPress, and Omeka S isolate author JavaScript from the platform's authenticated session?

- **RQ1.** Which integrations execute author JavaScript?
- **RQ2.** Which execute it in the same origin as the platform?
- **RQ3.** Which mitigations preserve interactivity and tracking without exposing the authenticated DOM?
- **RQ4.** What risks does AI-generated HTML/JS introduce in educational contexts?

2. Background

> *Stub (expand from ES Section 2):* Same-Origin Policy; iframe `sandbox` and the **propagation of sandbox flags to nested iframes** (why a nested YouTube player breaks under an opaque sandbox); CSP; SCORM/H5P/eXeLearning trust models.

3. Methodology

> *Stub (expand from ES Section 3):* analysed stable versions + commits (Moodle 5.0.7 core `2104c372962`; `mod_exelearning 2c5473d`; `mod_exeweb 60d24fb`; `mod_exescorm e985f4d`; eXeLearning editor `8101f54e`; WordPress via `wp-env`; Omeka S via Docker `erseco/alpine-omeka-s:develop`; `wp-franer 7fbf694`), disposable Docker environment, **two browser engines (Chromium and Firefox/Gecko 146 via Playwright)**, risk-classification criteria, and ethics. Note: the **secure mode is the default design** of the maintained integrations, distinct from the analysed same-origin behaviour.

Probe (`probe.js`) — measure definitions. The PoCs share a probe that returns only booleans and redacted error names; it never reads real values, never makes network calls, never `POST`s, never invokes SCORM mutators.

Boolean	What it detects (never exercises)
`canRunJavascript`	the browser runs author script
`isOpaqueOrigin`	the document runs in an opaque origin (`null`)
`sandboxAttr` / `sandboxAllowsSameOrigin`	effective `sandbox` and whether it grants `allow-same-origin`
`canAccessParent` / `canReadParentDocument`	whether `window.parent/parent.document` are reachable (same origin)
`canReadParentCookie`	whether `parent.document.cookie` is readable (value always `REDACTED`)
`canFindSesskey`	whether the `sesskey`/nonce is present in the same-origin DOM (value `REDACTED`)
`canFindCourseEditForms` / `canFindCourseEditLinks`	presence of edit forms/links
`canAccessTop` / `canAttemptTopNavigation`	top-window reachability (does not navigate; `not_attempted`)
`canOpenPopups`	opens and immediately closes a 1×1 popup (harmless)
`canUsePostMessage` / `canPostMessageToParent`	channel availability (sends nothing)
`canCallScormApi` / `scormApiFlavor`	whether `window.API/API_1484_11` is reachable (does not invoke it)
`canUseLocalStorage` / `canUseSessionStorage`	same-origin storage access
`sandboxEscape`	**always `false`**: detected by design, never attempted

Risk classification. *Low:* no author JS, or isolated (opaque/`srcdoc`) with no parent access. *Medium:* isolated JS with residual channels (popups, `postMessage`) or evadable semantic filtering. *High:* author JS in the **same origin** as the LMS, or in the **top window** without sandbox.

4. Results

4.1 Main table

Platform / resource	Runs author JS?	Same origin as LMS?	Real isolation	Risk level
`mod_page` (Page)	**Yes** (`noclean=true`; verified)	Yes (top window, no iframe)	None server-side; only gated by `mod/page:addinstance`	**High** if a teacher edits it (runs in every viewer's session)
`mod_scorm` (core)	Yes	Yes	None (no sandbox)	High
`mod_h5pactivity` / `core_h5p`	Params: **No** (filtered). Libraries: **Yes** (`preloadedJs`, trusted code)	Yes	Params filtered by semantics; library JS runs *same-origin*, unsandboxed	Low for content; **high** if libraries can be installed (`h5p:updatelibraries`, manager/admin)
`mod_exelearning`	Yes	**Configurable** (`secure`=opaque default / `legacy`=same-origin)	Strong in `secure` (opaque origin + bridge), partial in `legacy`	Low in `secure` / medium-high in `legacy`
`mod_exeweb`	Yes	Yes	None	High
`mod_exescorm`	Yes	Yes	None	High
`wp-exelearning`	Yes	**Configurable** (`secure`=opaque default / `legacy`=same-origin)	Strong in `secure` (opaque origin), partial in `legacy`	Low in `secure` / medium-high in `legacy`
`omeka-s-exelearning`	Yes	**Configurable** (`secure`=opaque default / `legacy`=same-origin)	Strong in `secure` (opaque origin), partial in `legacy`	Low in `secure` / medium-high in `legacy`
`wp-franer` (reference)	Yes, isolated	**No** (opaque `srcdoc`) + CSP	Strong	Low

4.2 `mod_page` — protection is capability, not sanitisation

> *Stub (expand from ES Section 4.2):* verified `noclean=true` execution in the top window, in every viewer's session; protection = `mod/page:addinstance`, not sanitisation; impact in a student session (HttpOnly cookie not readable but session-ridden; profile changes; worm via `core_message_send_instant_messages`, click-dependent); origin-confined but a latent escalation vector.

Dormant, targeted, patient execution (add). A same-origin payload **need not act immediately**. It can lie **dormant** —doing nothing visible to students— and **fingerprint the viewer** (`M.cfg.userId`/roles, DOM elements only managers see, or capability probing) to **fire the privileged payload only when an administrator/manager opens the resource**. This makes the attack **patient and targeted**: invisible during normal use, it waits for the highest-privilege viewer, raising both the probability of success and the impact. It can also **actively lure** the privileged viewer: the worm's messaging channel (`core_message_send_instant_messages`) can send a **bait message to a higher-privilege user** (subject to messaging policy: contacts/coursemates by default, anyone if `messagingallusers` is on). Once such a viewer opens it, impact is **bounded by their capabilities**: a course creator/manager could **create a course** (reproduced via `course/edit.php` form-scraping) and **enrol learners** (`enrol_manual_enrol_users`); an administrator could alter accounts/site config. Corollary for defenders: **absence of symptoms is not evidence of safety.**

4.3 Native SCORM (`mod_scorm`)

> *Stub (expand from ES Section 4.3):* SCO walks `window.parent`; iframe without `sandbox`; server-side defence (`confirm_sesskey` + capability); least client-isolated; client-side score tampering [2].

4.4 H5P (`mod_h5pactivity` / `core_h5p`)

H5P writes content into an `about:blank` iframe that **inherits the parent origin** (not opaque, **no** `sandbox`), so isolation does not come from the origin. Two planes must be distinguished.

Parameter plane (negative control). `content.json` text is filtered by `H5PContentValidator`: `validateText`/`filter_xss` use a closed tag allowlist with **no `<script>`** and drop `on*` attributes (`h5p.classes.php:4303-4384`, `:5033-5054`);

fields without `tags` are `htmlspecialchars`-escaped. Our `evil.h5p` injects `<script>`/`<img onerror>` into a text field and H5P discards them — a **negative control**.

Library plane (protection is capability, not sanitisation). But H5P **libraries are trusted code by design**: their `preloadedJs` loads as `<script src=pluginfile.php/…/core_h5p/…>` and runs **same-origin, unsandboxed** in the Moodle page (`player.php:484-500`, `h5p.js:391-437`). H5P states it plainly — *"JavaScript files … are by default and necessity allowed for H5P libraries but not for H5P content"* — and warns that *"only trusted users should be given permission to update h5p libraries"* [3]. The only barrier to author content shipping its **own** library is a **capability**, not a filter: installing a new library from an uploaded `.h5p` requires `moodle/h5p:updatelibraries` (archetype **manager**, `RISK_XSS`), evaluated **against the uploader** (`api.php:403-405`, `helper.php:210-224`, `h5p.classes.php:1577-1579`). Teachers hold only `moodle/h5p:deploy` (reuse installed libraries, not install new ones); a package needing an absent library is **rejected at validation**. We built `evil-h5p-library.h5p` (library `H5P.ExePocAlert`, whose `preloadedJs` shows a notice + read-only capability booleans): deployed with the manager capability it **runs author JavaScript in the LMS origin**. Same pattern as `mod_page`: the real defence is **capability/role**, not sanitisation or a sandbox — an **admin-trust / supply-chain** risk. Real-world precedent: importing a malicious `.h5p` reached **authenticated RCE** in Chamilo (`CVE-2026-30875`) [4].

H5P is not immune on the content side either: documented evadable XSS (`MDL-67110` [5], `CVE-2024-43439` reflected via H5P error message [6], `CVE-2024-3111` stored XSS via SVG upload to backdoor in the WordPress H5P plugin [7]); and its `postMessage` validates `event.source`/`context` but **not** `event.origin`, posting with `'*'` — the check [8] found exploitable on 84 popular sites.

> **Nuanced verdict:** H5P does **not** run author HTML/JS from *parameters* (it filters them: negative control), but **libraries** are trusted code running *same-origin*, unsandboxed; what separates author content from executing JS is the `moodle/h5p:updatelibraries` capability (manager/admin), not sanitisation. Same pattern as `mod_page`. Evidence: `evidencias/resultados-h5p-library.json`; PoC: `poc/evil-h5p-library.h5p`.

4.5 eXeLearning in Moodle

> *Stub (expand from ES Section 4.5):* stable `sandbox` keeps `allow-same-origin` for the synchronous SCORM bridge → not truly isolated; `mod_exeweb`/`mod_exescorm` have no sandbox; verified `legacy` reads `parent.M.cfg.sesskey` and forges `core_user_update_users`.

4.6 eXeLearning in WordPress and Omeka S

> *Stub (expand from ES Section 4.6):* stable same-origin sandboxes; WP REST `__return_true` unauthenticated content read; nonces/CSRF tokens, same logic as `sesskey`.

4.7 `wp-franer` (isolation reference)

> *Stub (expand from ES Section 4.7):* `srcdoc` opaque + minimal sandbox + injected restrictive CSP + validated `postMessage` + parent-side authenticated fetch. **Transparency: `wp-franer` is the author's own reference implementation; see Section 9.**

5. Discussion

> *Stub (expand from ES Section 5):* implications for teachers and administrators; generative-AI/OER impact (RQ4, CEDEC/INTEF); compatibility-vs-security dilemma (opaque origin breaks third-party players); the **dormant/targeted threat** means visible normal behaviour does not imply safety.

6. Mitigations

6.1 External state: author-trust model

> *Stub:* Moodle core media filter (provider allowlist, direct embed, no sandbox); H5P curated libraries; SCORM trusted package.

6.2 Implemented mitigation: the eXeLearning secure mode (own contribution)

Stub (expand from ES Section 6.2): opaque origin by default; single source of truth for sandbox tokens; **response-level `sandbox` CSP directive**; restrictive CSP (`connect-src 'self'`, `frame-ancestors`, `object-src 'none'`, `base-uri 'none'`, Permissions-Policy); SVG/XML neutralisation; no direct parent iframe access (server-side or validated `postMessage`, closed action list, SCORM score only, credentials only in parent); read-only file capability (`tokenpluginfile`); security tests.

Threat table (asset → condition → impact → mitigation):

Asset	Threat	Condition	Impact	Mitigation
LMS session	same-origin JS rides the session	resource runs author JS in the LMS origin	authenticated actions (per viewer role)	opaque origin (no `allow-same-origin`)
Authenticated DOM	parent read from the iframe	`allow-same-origin` present	data/nonce exposure	sandbox without same-origin + response-level `sandbox`
SCORM tracking	client-side score tampering	JS API reachable same-origin	falsified grades	validated `postMessage` bridge + server-side re-validation
File token	exfiltration of the read-only token	CSP allows `img/script-src https:`	temporary package-file access	strict-CSP profile (proposed, Section 6.3) + short TTL
Privileged viewer	dormant, targeted, active payload	author JS waits for —or **lures via a bait message**— an admin/manager to open it	course creation, enrolment, site control	opaque origin removes the foothold; content review by role
Other users	bait-message worm	`core_message_send_instant_messages` (role `user`)	viewer-independent propagation	content origin-confined; review by role

6.3 Proposed mitigations (future work)

Stub (expand from ES Section 6.3): **(a)** allow video from any provider without an allowlist via a **structural cross-origin invariant** (https + cross-origin to the LMS; reject same-site/subdomain/IP/loopback/userinfo) — a cross-origin iframe cannot read the LMS, the same trust model Moodle already uses for YouTube; alternative: server-side oEmbed [9]. **(b)** optional **strict-CSP profile** dropping `https:` from `img/script/media-src` to close file-token exfiltration. **(c)** admin-configurable embed helper across the three integrations.

7. Limitations

Stub: local disposable environment; specific versions/commits; no destructive exploitation; two browser engines verified (Chromium and Firefox/Gecko), not exhaustive across all browsers; client-isolation threat model.

8. Ethics and responsible disclosure

Stub (expand from ES Section 8): documented, by-design behaviours (not third-party 0-days); secure mode is an already-integrated mitigation; redacted PoCs, no reusable payloads, reversible lab changes reverted; coordinate with maintainers for any unpatched third-party finding.

9. Conflict of interest

Stub (expand from ES Section 9): the author is a collaborator of the eXeLearning project and author/maintainer of several evaluated pieces —`mod_exelearning`, `wp-exelearning`, `omeka-s-exelearning` (secure mode, Section 6.2) and `wp-franer` (the secure-execution reference, Section 4.7)— disclosed for transparency; results are verifiable from source and artifacts. **Third-party software with no author ties: Moodle (core, `mod_page`/`mod_scorm`), SCORM, and H5P.**

10. Artifact availability

A reproducible artifact bundle is published at **https://github.com/erseco/lms-untrusted-content-security-paper** (paper text under **CC-BY-4.0**; code and PoCs under **MIT**): the `probe.js` probe (15 redacted

checks), the PoC packages + `build.sh` — including the H5P library `evil-h5p-library.h5p` that demonstrates `preloadedJs` execution —, the full `file:line` matrix (`matriz-seguridad.md`), per-platform/per-browser appendices (`anexos-tecnicos.md`), JSON evidence (`evidencias/`, including the Firefox cross-browser checks of the three integrations with their Playwright scripts, and `resultados-h5p-library.json`), and the document-generation script. Analysed versions are identified by the commits in Section 3. The PoCs contain no reusable payloads; copyrighted source PDFs are not redistributed (linked by DOI/URL).

11. Conclusions

> *Stub (expand from ES Section 11):* answer RQ1–RQ4; the verified hardening pattern (opaque origin + strict sandbox + CSP incl. response directive + validated `postMessage`) is already the default of the maintained integrations; generative AI makes the risk routine. Rule: **isolate, validate, and do not trust the resource's JavaScript as if it were part of the platform.**

12. Generative AI use statement

Generative AI tools (LLM-based writing and coding assistants) were used in preparing this work to support drafting and restructuring the text, generating and refactoring the proof-of-concept and evidence scripts, and producing tables. **AI is not listed as an author.** The author designed the research, defined the methodology, ran and verified every test in the lab environment, manually checked each technical claim, the code, and the cited evidence, and takes **full responsibility** for the content.

13. References

[1] CEDEC (Centro Nacional de Desarrollo Curricular en Sistemas no Propietarios), "Mantener la «a» de abierto en los REA en tiempos de IA." Accessed: June 14, 2026. [Online]. Available: https://cedec.intef.es/mantener-la-a-de-abierto-en-los-rea-en-tiempos-de-ia/

[2] P. Hutchison, "Cheating in SCORM." Accessed: June 13, 2026. [Online]. Available: https://pipwerks.com/2009/03/22/cheating-in-scorm/

[3] H5P, "Security (H5P documentation): Libraries are trusted code." Accessed: June 14, 2026. [Online]. Available: https://h5p.org/documentation/installation/security

[4] MITRE CVE, "CVE-2026-30875: Authenticated remote code execution in chamilo LMS via H5P package import." Accessed: June 14, 2026. [Online]. Available: https://github.com/chamilo/chamilo-lms/security/advisories/GHSA-mj4f-8fw2-hrfm

[5] Moodle Tracker, "MDL-67110: XSS in malicious seemingly H5P content." Accessed: June 13, 2026. [Online]. Available: https://tracker.moodle.org/browse/MDL-67110

[6] Deutsche Telekom Security and Moodle, "CVE-2024-43439: Reflected XSS in Moodle via H5P error message." Accessed: June 13, 2026. [Online]. Available: https://github.security.telekom.com/2024/08/reflected-xss-moodle.html

[7] CleanTalk PSC, "CVE-2024-3111: Interactive content – H5P (WordPress) stored XSS to backdoor creation." Accessed: June 13, 2026. [Online]. Available: https://research.cleantalk.org/cve-2024-3111/

[8] S. Son and V. Shmatikov, "The postman always rings twice: Attacking and defending postMessage in HTML5 websites," in *Proceedings of the network and distributed system security symposium (NDSS)*, 2013. Available: https://www.ndss-symposium.org/ndss2013/ndss-2013-programme/postman-always-rings-twice-attacking-and-defending-postmessage-html5-websites/

[9] oEmbed, "oEmbed — an open format for embedding content via a provider API." https://oembed.com/.

[10] MDN Web Docs, "The inline frame element: The sandbox attribute." Accessed: June 13, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTML/Element/iframe#sandbox

[11] MDN Web Docs, "Same-origin policy." Accessed: June 13, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/Security/Same-origin_policy

[12] MDN Web Docs, "Content security policy (CSP)." Accessed: June 13, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP

[13] MDN Web Docs, "Window: postMessage() method." Accessed: June 13, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Window/postMessage

[14] MDN Web Docs, "Permissions-policy header." Accessed: June 13, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Permissions-Policy

[15] R. Lyne and M. West, "SameSite cookies explained." Accessed: June 13, 2026. [Online]. Available: https://web.dev/articles/samesite-cookies-explained

[16] MDN Web Docs, "Using HTTP cookies: Restrict access with HttpOnly." Accessed: June 13, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Cookies

[17] WHATWG, "HTML living standard — sandboxing." Accessed: June 13, 2026. [Online]. Available: https://html.spec.whatwg.org/multipage/browsers.html#sandboxing

[18] Advanced Distributed Learning (ADL), "SCORM 1.2 run-time environment," Advanced Distributed Learning Initiative, 2001. Accessed: June 13, 2026. [Online]. Available: https://adlnet.gov/projects/scorm/

[19] Advanced Distributed Learning (ADL), "SCORM 2004 4th edition — run-time environment (API API_1484_11)," Advanced Distributed Learning Initiative, 2009. Accessed: June 13, 2026. [Online]. Available: https://adlnet.gov/projects/scorm/

[20] H5P Group, "H5P documentation — content types and libraries." Accessed: June 13, 2026. [Online]. Available: https://h5p.org/documentation

[21] Moodle, "Page resource." Accessed: June 13, 2026. [Online]. Available: https://docs.moodle.org/en/Page_resource

[22] Moodle Developer Docs, "Output functions: format_text and trusted text." Accessed: June 13, 2026. [Online]. Available: https://moodledev.io/docs/apis/subsystems/output

[23] Moodle Developer Docs, "Security guidelines." Accessed: June 13, 2026. [Online]. Available: https://moodledev.io/general/development/policies/security

[24] OWASP Foundation, "Cross site scripting (XSS)." Accessed: June 13, 2026. [Online]. Available: https://owasp.org/www-community/attacks/xss/

[25] OWASP Foundation, "Clickjacking defense cheat sheet." Accessed: June 13, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Clickjacking_Defense_Cheat_Sheet.html

[26] E. Z. Yang, "HTML purifier — standards-compliant HTML filtering." Accessed: June 13, 2026. [Online]. Available: http://htmlpurifier.org/

[27] P. Hutchison, "SCORM API wrapper for JavaScript (pipwerks)." Accessed: June 13, 2026. [Online]. Available: https://github.com/pipwerks/scorm-api-wrapper

[28] A. Khamparia and B. Pandey, "Threat driven modeling framework using petri nets for e-learning system," *SpringerPlus*, vol. 5, no. 1, p. 446, 2016, doi: 10.1186/s40064-016-2101-0.

[29] R. R. Al-azaiza and T. S. M. Barhoom, "Enhance MOODLE security against XSS vulnerabilities," *International Journal of Computing and Digital Systems*, vol. 5, no. 5, pp. 421–430, 2016, doi: 10.12785/ijcds/050507.

[30] A. J. J. Joseph and M. Mariappan, "Approaches to overcome security risks and threats in online learning applications," in *Secure data management for online learning applications*, CRC Press, 2023. doi: 10.1201/9781003264538-3.

[31] Sonar, "Playing dominos with Moodle's security." Accessed: June 13, 2026. [Online]. Available: https://www.sonarsource.com/blog/playing-dominos-with-moodles-security-2/

[32] A. Barth, C. Jackson, and J. C. Mitchell, "Securing frame communication in browsers," *Communications of the ACM*, vol. 52, no. 6, pp. 83–91, 2009, doi: 10.1145/1516046.1516066.

[33] S. Stamm, B. Sterne, and G. Markham, "Reining in the web with content security policy," in *Proceedings of the 19th international conference on world wide web (WWW)*, 2010, pp. 921–930. doi: 10.1145/1772690.1772784.

[34] L. Weichselbaum, M. Spagnuolo, S. Lekies, and A. Janc, "CSP is dead, long live CSP! On the insecurity of whitelists and the future of content security policy," in *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security (CCS)*, 2016, pp. 1376–1387. doi: 10.1145/2976749.2978363.

[35] J. Weinberger, P. Saxena, D. Akhawe, M. Finifter, R. Shin, and D. Song, "A systematic analysis of XSS sanitization in web application frameworks," in *Computer security – ESORICS 2011*, in Lecture notes in computer science. Springer, 2011, pp. 150–171. doi: 10.1007/978-3-642-23822-2_9.

[36] P. Agten, S. Van Acker, Y. Brondsema, P. H. Phung, L. Desmet, and F. Piessens, "JSand: Complete client-side sandboxing of third-party JavaScript without browser modifications," in *Proceedings of the 28th annual computer security applications conference (ACSAC)*, 2012, pp. 1–10. doi: 10.1145/2420950.2420952.

[37] P. Dawson, *Defending assessment security in a digital world: Preventing e-cheating and supporting academic integrity in higher education*. Routledge, 2020. doi: 10.4324/9780429324178.

[38] eXeLearning / Área de Tecnología Educativa, *Mod_exelearning: Secure iframe mode (opaque-origin sandbox + SCORM postMessage bridge)*. (2026). Accessed: June 14, 2026. [Online]. Available: https://github.com/ateeducacion/mod_exelearning

[39] eXeLearning / Área de Tecnología Educativa, *Wp-exelearning: Secure (opaque-origin) content iframe mode*. (2026). Accessed: June 14, 2026. [Online]. Available: https://github.com/exelearning/wp-exelearning

[40] eXeLearning / Área de Tecnología Educativa, *Omeka-s-exelearning: Secure (opaque-origin) preview iframe mode*. (2026). Accessed: June 14, 2026. [Online]. Available: https://github.com/exelearning/omeka-s-exelearning

[41] N. Nikiforakis *et al.*, "You are what you include: Large-scale evaluation of remote JavaScript inclusions," in *Proc. 2012 ACM conference on computer and communications security (CCS '12)*, 2012, pp. 736–747. doi: 10.1145/2382196.2382274.

[42] T. Lauinger, A. Chaabane, S. Arshad, W. Robertson, C. Wilson, and E. Kirda, "Thou shalt not depend on me: Analysing the use of outdated JavaScript libraries on the web," in *Proc. 2017 network and distributed system security symposium (NDSS '17)*, 2017. doi: 10.14722/ndss.2017.23414.

[43] S. Roth, T. Barron, S. Calzavara, N. Nikiforakis, and B. Stock, "Complex security policy? A longitudinal analysis of deployed content security policies," in *Proc. 2020 network and distributed system security symposium (NDSS '20)*, 2020. doi: 10.14722/ndss.2020.23046.

[44] M. Steffens, C. Rossow, M. Johns, and B. Stock, "Don't trust the locals: Investigating the prevalence of persistent client-side cross-site scripting in the wild," in *Proc. 2019 network and distributed system security symposium (NDSS '19)*, 2019. doi: 10.14722/ndss.2019.23009.

[45] B. Stock, S. Lekies, T. Mueller, P. Spiegel, and M. Johns, "Precise client-side protection against DOM-based cross-site scripting," in *Proc. 23rd USENIX security symposium (USENIX security '14)*, 2014, pp. 655–670. Available: https://www.usenix.org/system/files/conference/usenixsecurity14/sec14-paper-stock.pdf

[46] M. Heiderich, J. Schwenk, T. Frosch, J. Magazinius, and E. Z. Yang, "mXSS attacks: Attacking well-secured web-applications by using innerHTML mutations," in *Proc. 2013 ACM SIGSAC conference on computer and communications security (CCS '13)*, 2013, pp. 777–788. doi: 10.1145/2508859.2516723.

[47] P. Wang, B. Á. Gumundsson, and K. Kotowicz, "Adopting trusted types in production web frameworks to prevent DOM-based cross-site scripting: A case study," in *2021 IEEE european symposium on security and privacy workshops (EuroS&PW)*, 2021, pp. 60–73. doi: 10.1109/EuroSPW54576.2021.00013.

[48] S. A.-L. Akacha and A. I. Awad, "Enhancing security and sustainability of e-learning software systems: A comprehensive vulnerability analysis and recommendations for stakeholders," *Sustainability*, vol. 15, no. 19, p. 14132, 2023, doi: 10.3390/su151914132.